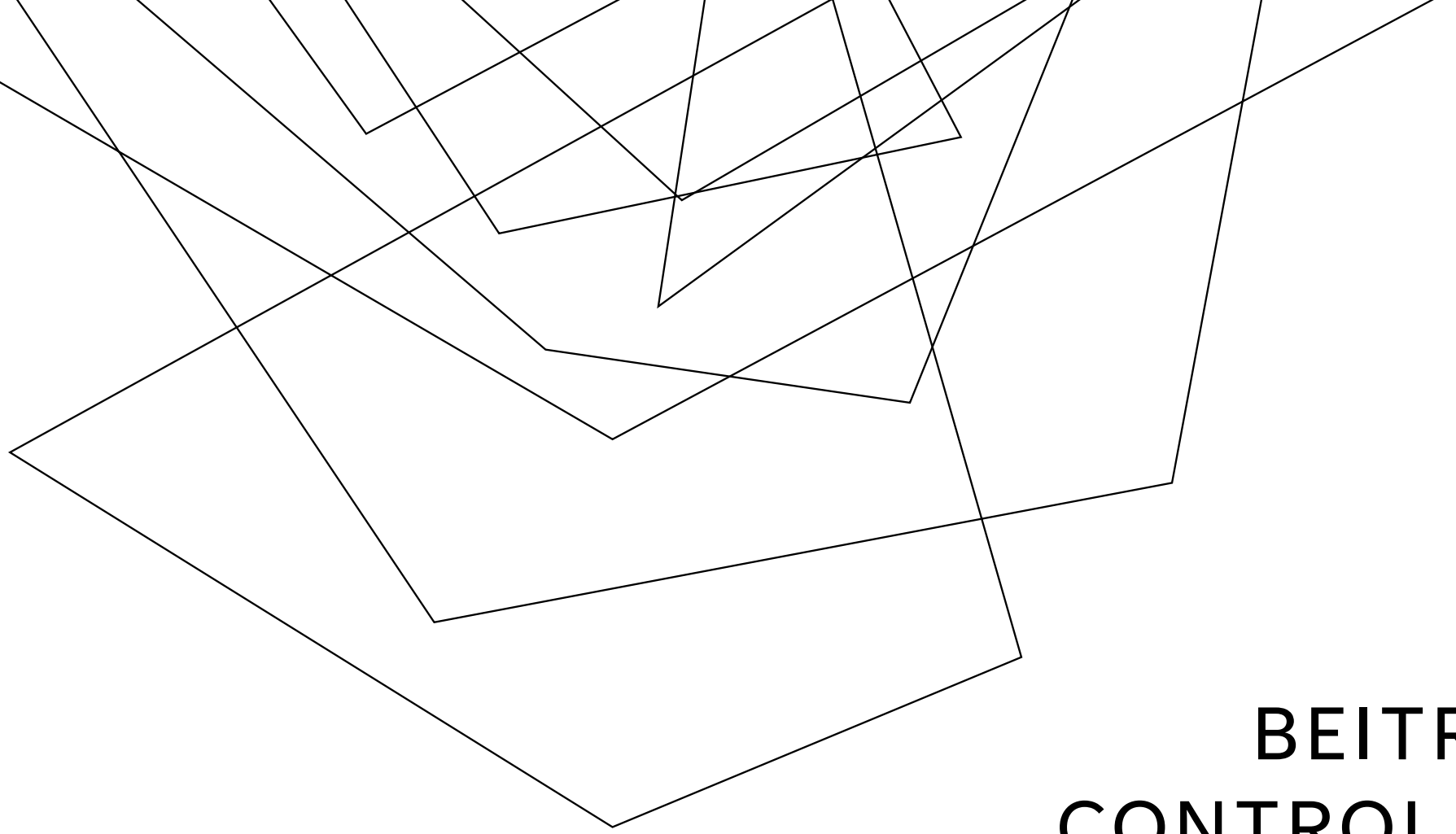


BEITRAG CONTROL/SOLVER

ROBOTIK PROJEKTSTUDIUM
PROF. DR. FISCHER - SS24

GLIEDERUNG

- Beitrag Team Control
  - Inverse Kinematik
  - ROS2 Integration
- Beitrag Team Solver
  - Layout Optimierung
  - ROS2 Integration

GLIEDERUNG

- Beitrag Team Control
  - Inverse Kinematik
  - ROS2 Integration
- Beitrag Team Solver
  - Layout Optimierung
  - ROS2 Integration

BEITRAG TEAM CONTROL

INVERSE KINEMATIK

INVERSE KINEMATIK

Implementierung der Newton-Raphson Methode

1. Newton-Raphson Methode bereits bekannt
2. Implementierung dieser in der Roboter-Klasse
3. Benötigt Startpose Tep und Startkonfiguration q0
4. Gibt zurück:
   a. Fehlerwert
   b. Gelenkkonfiguration
   c. Status (Fehler oder Erfolg)

```
def ikine_NR(self, Tep: SE3,
             q0: NDArray,
             eps: float = 1e-3,
             i_lim: int = 100,
             We: NDArray = np.ones((6,))
             ) -> Tuple[float, NDArray, bool]:
```

BEITRAG TEAM CONTROL

ROS2 INTEGRATION

ROS2 INTEGRATION

Aufgaben

1. Empfangen der Spawn- und Zielpose von Solver-Node
2. Berechnen der Trajektorien, um Teil-Zielpose zu erreichen
3. Publishen der Gelenkkonfigurationen an Visualizer-Node
4. Publishen des Puzzle-Status (Abgelegt oder Angeheftet) an Solver-Node

ROS2 INTEGRATION

Schnittstellen

1. Topic: „/solver/nextpiece“

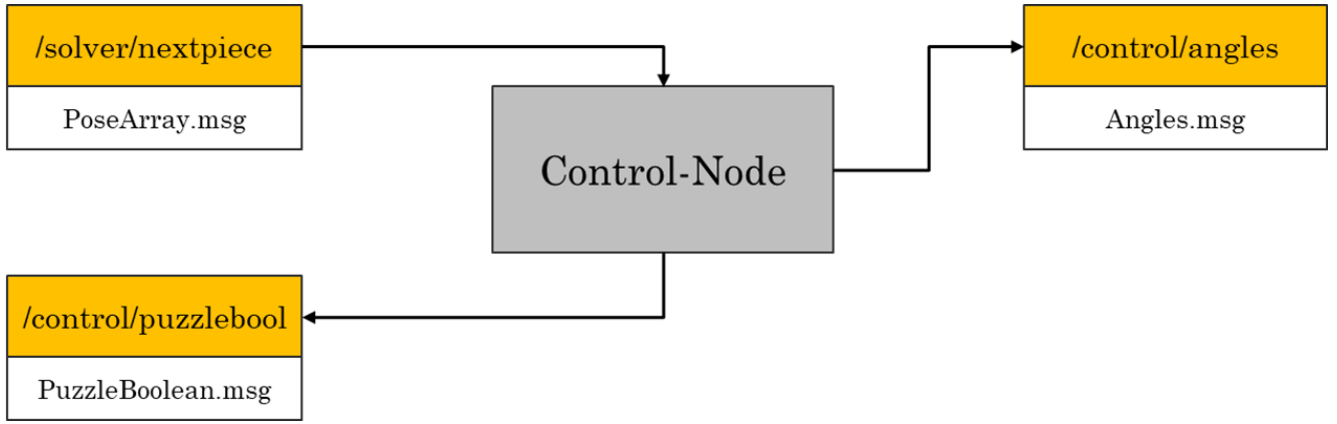
   ⇒ Spawn- und Zielpose

2. Topic: „/control/puzzlebool“

   ⇒ Status des aktuellen Puzzleteils

3. Topic: „/control/angles“

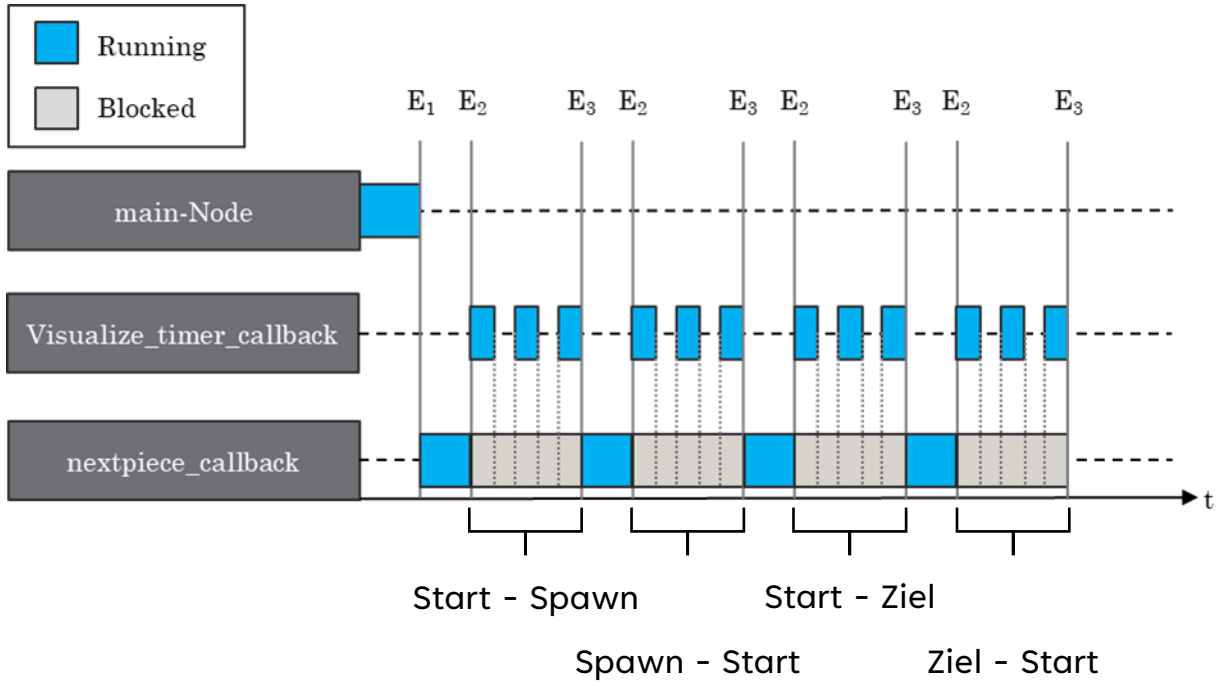
   ⇒ Gelenkkonfiguration und Greifinformation



ROS2 INTEGRATION

Ablauf

1. E_1: Nextpiece-Nachricht empfangen
2. E_2: Teiltrajektorie berechnet
3. E_3: Trajektorie versendet



GLIEDERUNG

- Beitrag Team Control
  - Inverse Kinematik
  - ROS2 Integration
- Beitrag Team Solver
  - Layout Optimierung
  - ROS2 Integration

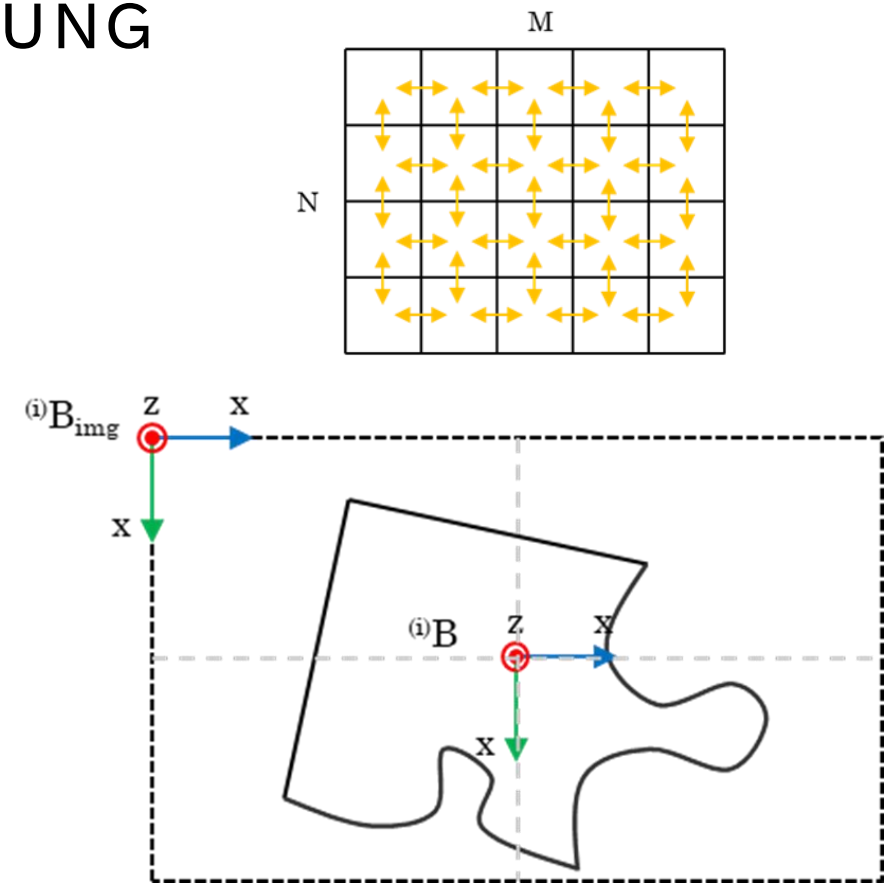
BEITRAG TEAM SOLVER

LAYOUT OPTIMIERUNG

(LINEARE) LAYOUT OPTIMIERUNG

Gegeben

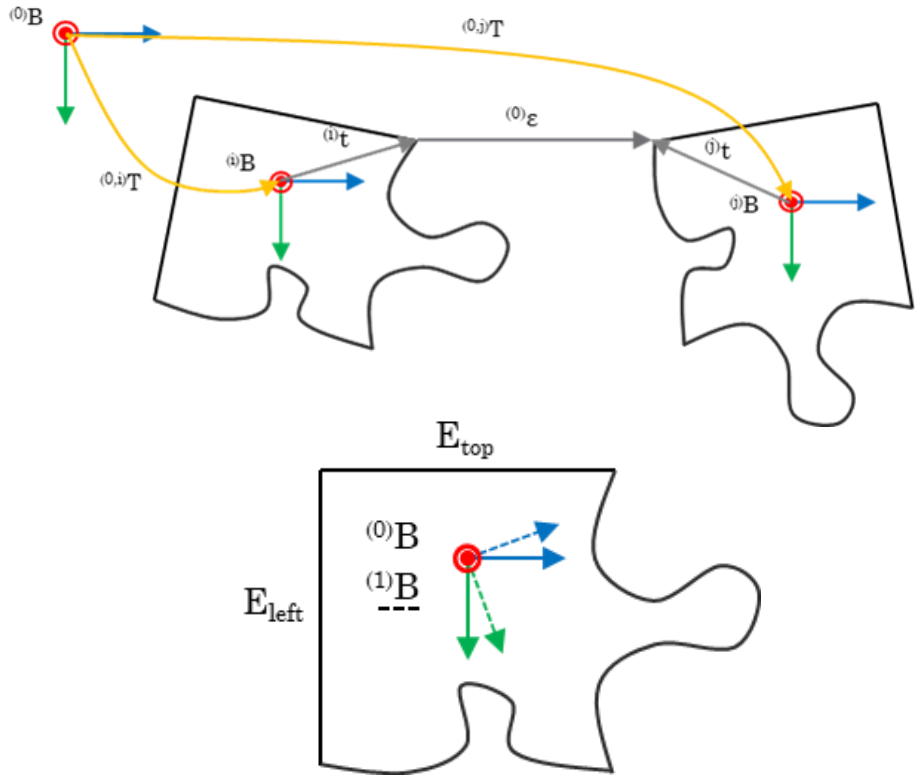
1. Gegeben das gelöste Puzzle (Kanten/ Ecken-Korrespondenzen)
2. Koordinatensysteme der Puzzleteile befinden sich in der Mitte des Bildes (reine Translation)



(LINEARE) LAYOUT OPTIMIERUNG

Ziel

1. Finden der Posen der Puzzleteile ${}^{(0,i)}T$ im Referenzsystem ${}^{(0)}B$, so dass:
   a. Die quadratische Summe der euklidischen Distanz der Fehlervektoren zwischen korrespondierendem Ecken minimal wird
   b. Referenzsystem in Ursprung der linken, oberen Puzzleteil-Basis ${}^{(1)}B$
   c. Kanten des linken, oberen Teils entlang der x- bzw. y-Achse von ${}^{(0)}B$



(LINEARE) LAYOUT OPTIMIERUNG

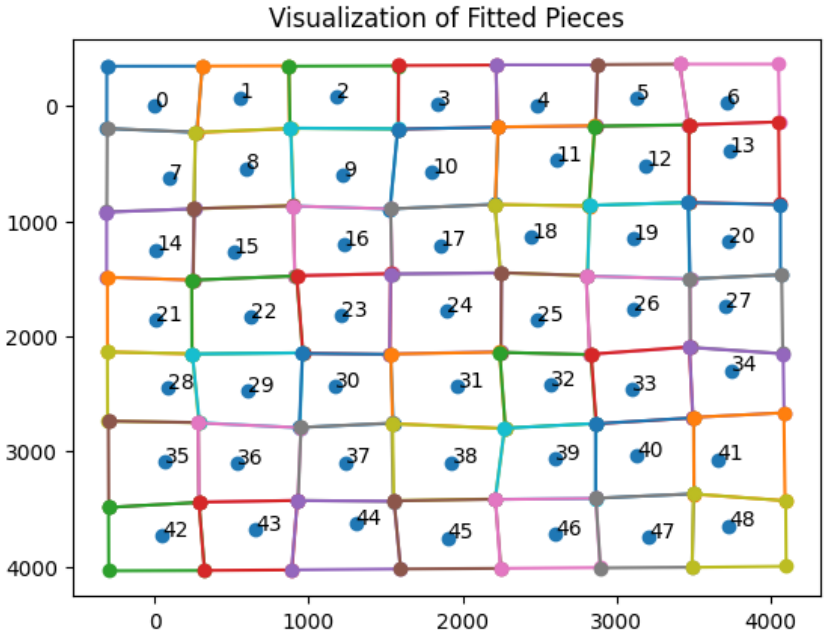
Formulierung des Problems

1. Least Squares Problem
2. Berücksichtigung von b) und c) durch zusätzliche Fehlerterme
3. Linearisierung der Rotation nach dem Winkel erlaubt effiziente Lösung des Problems über SVD
4. Mehrere Iterationen nötig, da Linearisierungsfehler

$$\arg\min_x \frac{1}{2} \sum^{|K|} \|\epsilon(x)\|_2^2$$

$$x = \begin{bmatrix} \phi_i & x_i & y_i & \dots & \phi_{|P|} & x_{|P|} & y_{|P|} \end{bmatrix}^T$$

(LINEARE) LAYOUT OPTIMIERUNG



Ergebnisse

1. Konvergenz nach 30 ms (3 Iterationen)

2. Optisch werden sehr gute Ergebnisse erreicht

3. Einheit der Posen Translation ist Pixel!



BEITRAG TEAM SOLVER

ROS2 INTEGRATION

ROS2 INTEGRATION

Aufgaben

1. Empfangen der segmentierten Bilder von Digitalize-Node
2. Lösen des Puzzles
3. Publishen der Spawnpose des Puzzles an Visualizer-Node
4. Publishen der Spawn- und Zielpose des Puzzles an Control-Node

ROS2 INTEGRATION

Schnittstellen

1. Action Client: „/digitalize/segement“

   ⇒ Bilder (+ ID)

2. Topic „/contol/puzzlebool“

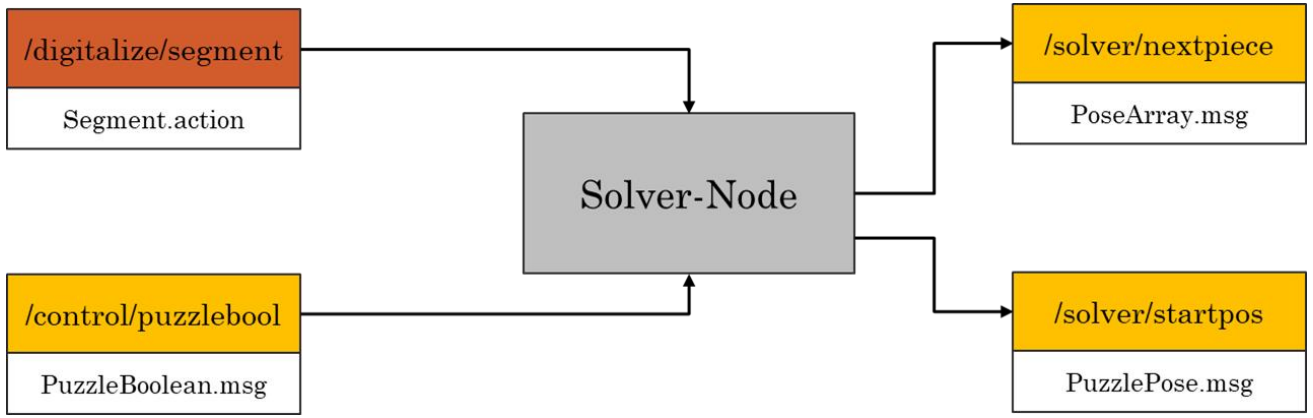
   ⇒ Status des aktuellen Puzzleteils

3. Topic „/solver/nextpiece“

   ⇒ Spawn- und Zielpose (Translation in m)

4. Topic „/solver/startpos“

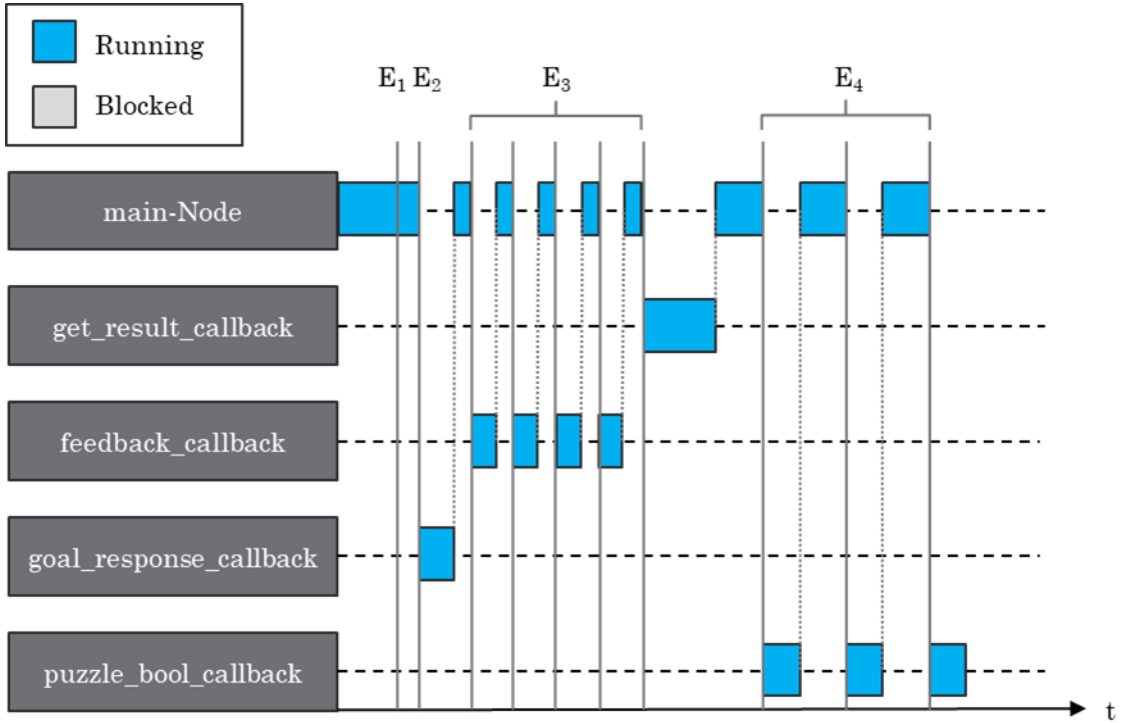
   => Spawnpose (Translation in mm)



ROS2 INTEGRATION

Ablauf

1. E_1: Action-Client Goal-Anfrage
2. E_2: Antwort des Action-Servers
3. E_3: Feedback-Nachricht (Bild) kommt an
4. E_4: Puzzle Teil wurde abgelegt (PuzzleBoolean data toggelt auf False)



VIELEN DANK FÜR IHRE AUFMERKSAMKEIT!

Sami Hamza

sami.hamza0@hm.edu